

Documentación Técnica - EdgeLeak AI (Fase 2: Seguridad, SMTP y TDD)

1. Evolución de la Arquitectura de Datos (Migración a v3)

Para soportar las políticas de seguridad de cambio obligatorio de contraseña, el esquema relacional local (Sqlite) fue refactorizado y migrado a la versión 3 (edgeleak_v3.db).

El cambio fundamental radica en la tabla usuarios, a la cual se le añadió el campo lógico **es_temporal**. Dado que SQLite no posee un tipo de dato BOOLEAN nativo, este campo se implementó mediante un INTEGER (0 para Falso, 1 para Verdadero), permitiendo al sistema identificar si la sesión iniciada requiere una interrupción por motivos de seguridad.

2. Módulo de Recuperación de Credenciales (Protocolo SMTP)

El sistema integra una solución robusta de recuperación de acceso prescindiendo de servidores backend intermedios, conectándose directamente a la infraestructura de Google Workspace mediante el protocolo **SMTP (Simple Mail Transfer Protocol)**.

Características Técnicas de la Implementación:

- **Seguridad de Capa de Transporte:** Uso de SSL/TLS implícito para garantizar el cifrado de extremo a extremo durante la transmisión de la contraseña.
- **Autenticación S2S (Server-to-Server):** Implementación de "Contraseñas de Aplicación" de Google con Autenticación en 2 Pasos (2FA), aislando los permisos del correo principal (edgeleak.ai@gmail.com) y protegiendo las credenciales mediante el archivo de variables de entorno (.env).
- **Interfaz de Correo (HTML):** Los correos no se envían en texto plano; el servicio (EmailService) renderiza una plantilla HTML/CSS responsiva, mejorando la Experiencia de Usuario (UX) e indicando claramente la naturaleza transaccional y urgente del mensaje.

3. Algoritmo de Generación de Contraseñas Seguras

Ante la solicitud de restablecimiento, el sistema no solicita una clave de parte del usuario, sino que actúa de forma proactiva ejecutando el método criptográfico generarContrasenaSegura().

El algoritmo opera en tres fases matemáticas utilizando la librería dart:math:

1. **Inyección Obligatoria:** Garantiza la inserción de al menos un carácter de cuatro conjuntos estandarizados: Mayúsculas [A-Z], Minúsculas [a-z], Números [0-9] y Símbolos especiales [!@#\$%&*~].
2. **Relleno Aleatorio:** Completa la cadena hasta alcanzar una longitud estándar de la industria (12 caracteres).
3. **Dispersión (Shuffle):** Transforma la cadena en un arreglo (List) y aplica una dispersión aleatoria a sus índices para evitar patrones predecibles (por ejemplo, que la mayúscula siempre esté de primera).

4. Flujo de Intercepción UX: Force Password Reset

Una de las características de ciberseguridad más avanzadas del MVP es el "Cambio Obligatorio de Contraseña". El flujo opera de la siguiente manera:

1. En el método login del AuthController, tras verificar el hash/texto de la contraseña, el sistema lee la

bandera es_temporal.

2. Si `es_temporal == 1`, el enrutador (AppRoutes) intercepta la navegación hacia el Dashboard y redirige al usuario a un entorno aislado (ChangePasswordScreen).
3. **Manejo del Ciclo de Vida (Lifecycle):** Para evitar colisiones en el hilo de renderizado de la interfaz de usuario (UI Thread) al intentar resetear los validadores de formularios mientras la pantalla aún se está dibujando en memoria, se implementó el método `WidgetsBinding.instance.addPostFrameCallback`. Esto garantiza que la lógica de validación reactiva inicie un milisegundo después de completar el primer frame gráfico, eliminando cuellos de botella y posibles caídas de la aplicación (Crashes).
4. La pantalla utiliza un widget `PopScope` con la propiedad `canPop: false` para secuestrar el botón de retroceso de Android/iOS, impidiendo que el usuario navegue al sistema sin antes actualizar su credencial.

5. Pruebas Automatizadas e Integración (TDD)

Bajo la metodología de **Desarrollo Guiado por Pruebas (TDD)**, se integró el ecosistema `flutter_test` para auditar el código estático, garantizando la mantenibilidad a largo plazo del Proyecto Integrador.

El entorno de pruebas se divide en dos enfoques de caja blanca:

- **Pruebas Unitarias (Unit Tests):** Auditan la lógica pura de los controladores. Por ejemplo, se desarrollaron tests que inyectan cadenas de texto (ej. "hola123" vs "EdgeLeak2024*!") comprobando que la máquina de estados `RegExp` rechace la primera y apruebe la segunda. Adicionalmente, evalúan que los cálculos matemáticos del simulador de caudal (`DashboardController`) respeten los límites operativos de "Normal", "Anomalía" y "Fuga".
- **Pruebas de Componentes (Widget Tests):** Simulan un entorno virtual de Flutter con un `WidgetTester`. Se desarrolló un script (`widget_test.dart`) que monta la pantalla de Login y verifica asíncronamente que los elementos interactivos (`TextFields` y Botones) existan en el árbol de widgets y estén listos para interactuar.
- **Manejo de Memoria en Tests:** Se aplicó el método `addTearDown()` para eliminar instancias activas de `Timer.periodic`, garantizando que la memoria de la máquina virtual de Dart (Dart VM) se libere tras cada ejecución y evitando las fugas de memoria (Memory Leaks) características de los entornos asíncronos en bucle.